

HW2 – Introsort with Memory Pool

Author: Rhilo Sotto
Course: COMPE496

1. Design and Logic

This project implements Introsort, a hybrid sorting algorithm combining three strategies:

1. QuickSort – used for general sorting due to its average-case efficiency.
2. HeapSort – used when recursion depth exceeds $2 * \log_2(n)$ to avoid QuickSort's $O(n^2)$ worst-case.
3. InsertionSort – used for small subarrays (≤ 16 elements), where it is faster than recursive sorts due to low overhead.

The algorithm begins with QuickSort, switches to HeapSort when the recursion depth limit is reached, and falls back to InsertionSort for small subarrays. The depth limit ensures the algorithm maintains $O(n \log n)$ performance in all scenarios.

2. Recursion Depth Calculation

The recursion depth limit is calculated as:

$$\text{max_depth} = 2 * \log_2(n)$$

where n is the size of the array. This ensures QuickSort does not degenerate into $O(n^2)$ on nearly sorted or reversed arrays. If the recursion exceeds this depth, HeapSort handles the remaining subarrays.

3. Role of InsertionSort

- Subarrays of size ≤ 16 are sorted with InsertionSort.
 - InsertionSort is efficient for small datasets because it avoids recursion overhead and performs fewer comparisons on tiny arrays.
 - This hybrid approach significantly improves performance for mixed-size data.
-

4. Memory Pool and Performance

The MemoryPool class manages memory efficiently:

- Pre-allocates memory chunks to avoid repeated heap allocations.
- Uses a LIFO free list to reuse previously freed blocks.
- Tracks allocations and deallocations via counters.

Observations:

- With a chunk size of 1024 integers, most test arrays fit in a single chunk.
 - Allocation/deallocation counters only reached a maximum of 2 for these tests, but memory reuse still occurred internally.
 - Using the pool reduces heap allocation overhead, which becomes more noticeable with larger arrays or repeated allocations.
 - The program can run both with and without the pool for comparison, demonstrating the memory and performance benefits.
-

5. Testing

Test cases include:

- Small arrays, large reversed arrays, duplicates, already sorted, empty arrays, single elements, and random arrays.

- All tests passed with and without the Memory Pool.
 - Edge cases (empty arrays, single elements, reversed arrays) were handled correctly.
-

6. Challenges and Optimization

Challenges:

- Linking all `.cpp` files correctly to avoid undefined references.
- Integrating the MemoryPool without breaking the option to run the sorting algorithms without it.
- Ensuring Introsort handled extreme cases like large reversed arrays efficiently.

Optimizations:

- Using InsertionSort for small subarrays reduces recursive overhead.
- The memory pool pre-allocates memory and reuses it efficiently.
- Recursion depth switching ensures consistent $O(n \log n)$ performance.

Further improvements:

- Adjusting pool chunk size to match expected data size could better demonstrate memory reuse.
 - Parallelizing QuickSort partitions for very large datasets could further reduce runtime.
-

7. Conclusion

This implementation of Introsort demonstrates:

- Hybrid sorting efficiently handling different input sizes and edge cases.

- MemoryPool improves memory management and reduces allocation overhead.
- All test cases passed, including small, large, and edge-case arrays.
- The program allows direct comparison of sorting with and without the memory pool, illustrating its performance benefits.